

17A – Build Skeleton Implementation Guide

Sprint 0, part one. The repository, build, module structure, migration framework, boundary enforcement, the three application shells, the shared engine library, local environment, continuous integration, and the hello module that proves the whole chain. Everything a module developer later inherits and never has to invent.

Field	Value
Document	17A – Build Skeleton Implementation Guide (implements 17 Solution Architecture v0.2.1 with the stack of 14 and the shells of 30)
Version	v0.1 – for the platform pair; Sprint 0 weeks 1–2 (8–18 September 2026)
Date	8 September 2026
Companion	19A Kernel Services Implementation Guide fills the kernel package this guide scaffolds. 17A leaves interfaces and stubs; 19A replaces the stubs.
Audience	The platform pair (two developers), the DevOps engineer, the till track lead. Written for the keyboard: file names, commands and code are literal.
Restated content	Marked <i>restated from doc X \$Y</i> ; the source documents remain authoritative for the decision, this guide for the build.

The one test that matters at the end of Sprint 0. A developer clones the repository, runs one command, and within fifteen minutes has PostgreSQL, the broker, object storage and the identity provider running locally, the backend serving the hello module's endpoint through a scope-checked, audited, event-publishing command, the web shell showing the hello screen in Sinhala, Tamil and English, and the boundary tests green. If any step needs a person to explain it, Sprint 0 is not done.

1. What you are building

Doc 17 fixed the shape: one Spring Boot deployable organised as Modulith modules in four layers, one PostgreSQL cluster with a schema per module, a transactional outbox, a React back office, a native Android till, and a shared engine library used by both (*restated from doc 17 §5–6, doc 14 §2*). Sprint 0 turns that into a repository in which the first module can be written on day one of Sprint 1.

Deliverable	Done when
Monorepo with the backend, web shell, till shell, shared engine, infrastructure definitions and documentation	Section 2 layout exists; one command builds everything
Backend skeleton: application, kernel package with interfaces, module descriptor convention, profiles	Boundary tests pass; profiles web, worker, ingest start
Migration framework: schema per module, roles, extensions, collations, baseline	Flyway migrates an empty database to baseline; each module location is picked up
Boundary enforcement in the build	Modulith verification and ArchUnit rules fail the build on violation (Section 5)
Web shell scaffold	Login, scope banner placeholder, module route groups, generated client pipeline, i18n loader, component library seed
Till shell scaffold	Android project builds; kiosk-capable; local database; hello screen; consumes the shared engine
Shared engine library	Kotlin JVM module with the interfaces of ADR-31, stub implementations, tests, consumed by backend and till
Local environment	docker compose with PostgreSQL 16, broker, object storage, identity provider, mail catcher; seeded dev realm
Continuous integration	Pipeline runs build, boundary tests, unit and integration tests, web and Android builds, produces images and artefacts
Hello module	Section 12: the template every module copies, exercised end to end

2. Repository layout

One repository (knoweb). Monorepo because the shared engine, the OpenAPI slices and the schema conventions cross the backend/web/till boundary daily; splitting them creates version drift the single-DevOps model cannot afford.

```
knoweb/
  README.md                # the fifteen-minute start (Section 13)
  Makefile                 # make up | down | build | test | migrate | seed | gen-clients
  .github/workflows/ci.yml # or .gitlab-ci.yml (Section 11)
  docs/                   # design documents (PDF) + ADRs as markdown mirrors
  infra/
    compose/              # local environment (Section 10)
    deploy/               # provider-neutral deployment (doc 35 fills)
  backend/
    settings.gradle.kts
    build.gradle.kts
    app/                  # the Spring Boot deployable
      src/main/java/lk/coopfed/knoweb/
        KnowebApplication.java
        kernel/           # layer 0 (19A fills implementations)
        m1party/ m2catalogue/ m3pricing/ m4trading/ m5inventory/
        m6pos/   m7customers/ m8reporting/ m9integration/ m10procurement/
        hello/         # the template module (Section 12)
      src/main/resources/
        application.yml application-web.yml application-worker.yml application-ingest.yml
        db/migration/kernel/ db/migration/m1party/ ... db/migration/hello/
        seed/              # permission catalogue, templates, config items (19A/21A)
        openapi/           # one slice per module: m1party.yaml ... hello.yaml
        i18n/              # message catalogue: en.json si.json ta.json
      src/test/java/lk/coopfed/knoweb/
        ArchitectureTests.java # Modulith + ArchUnit (Section 5)
  shared-engine/           # Kotlin JVM library (Section 9), consumed by app and till
  web/
    package.json vite.config.ts tsconfig.json
    src/
      shell/              # auth, scope, navigation, i18n, layout, components
      modules/m1party/ ... hello/
```

```

generated/          # clients generated from backend/app/.../openapi
till/
settings.gradle.kts
app/                # Android application (Kotlin, Compose)
core/ peripherals/ sync/ # library modules
agent-test/         # device-owner test harness
tools/
gen-clients.sh  check-i18n.sh  sbom.sh

```

Aspect	Design
Package naming	lk.coopfed.knoweb. where module is kernel, hello, m1party ... m10procurement. The prefix letter plus number keeps sort order equal to layer order in every tool.
Schema naming	One PostgreSQL schema per module with the same name without the m-prefix number where unambiguous: kernel, hello, party, security (both M1), catalogue, pricing, trading, inventory, pos, customers, reporting, integration (<i>restated from doc 21 §3</i> : M1 owns two schemas).
Branches	main protected; short-lived feature branches; squash merge; conventional commit messages (feat (m1) : ...).

3. Toolchain and versions

Component	Version	Note
Java	21 (LTS)	Temurin distribution; toolchain pinned in Gradle
Gradle	8.10+, Kotlin DSL	Version catalogue in <code>gradle/libs.versions.toml</code>
Spring Boot / Spring Modulith	3.3.x / 1.2.x	Modulith provides module verification, event publication registry and documentation
PostgreSQL	16	With <code>pg_trgm</code> , <code>btree_gist</code> , <code>pgcrypto</code> ; ICU collations available (build with ICU)
Flyway	10.x	Multiple locations; <code>outOfOrder=false</code> ; <code>validateOnMigrate=true</code>
Testcontainers	1.20+	PostgreSQL, RabbitMQ, MinIO, Keycloak containers in integration tests
ArchUnit	1.3+	Layer and package rules beyond Modulith
Kotlin	2.0.x	Shared engine and till
Android	Gradle plugin 8.5+, <code>compileSdk 34</code> , <code>minSdk 30</code> (Android 11)	Doc 14 §3.1 minimum platform
Node / package manager	20 LTS / pnpm 9	Web shell
React / TypeScript / Vite	18 / 5.5 / 5	Doc 30 §3
OpenAPI tooling	<code>openapi-generator</code> (Java server stubs), <code>orval</code> or <code>openapi-typescript</code> (web client)	OpenAPI-first: the slice is the source; controllers implement generated interfaces
Containers	OCI images via Jib (backend) and Docker (web static)	No Dockerfile for the backend; Jib builds reproducibly in Gradle

4. Backend skeleton

4.1 Application and profiles

One artefact, three profiles (*restated from doc 17 §5, doc 14 §1.3*). Profiles change which components start, never the code.

Profile	Starts	Does not start
web	HTTP endpoints (back-office API, edge sync API, driver API), OIDC resource server, scope filter, idempotency interceptor	Outbox relay, event consumers, scheduled jobs
worker	Outbox relay, broker consumers, scheduled jobs, rendering, notifications	HTTP endpoints except health and metrics
ingest	Edge sync ingestion consumers only (the burst role)	Everything else except health and metrics

```
# application.yml (excerpt)
spring:
  application.name: knoweb
  datasource:
    url: ${DB_URL}           # jdbc:postgresql://pgbouncer:6432/knoweb
    username: ${DB_USER}     # app_rw
    hikari.maximum-pool-size: ${DB_POOL:10}  # pool x instances < pooler ceiling (doc 14 1.3)
  flyway:
    schemas:
      kernel,hello,party,security,catalogue,pricing,trading,inventory,pos,customers,reporting,integration
    locations: classpath:db/migration/kernel,classpath:db/migration/hello,classpath:db/migration/m1party, ...
    user: ${MIGRATION_USER}  # separate role with DDL rights
  modulith:
    events.republish-outstanding-events-on-restart: false  # our own outbox, not Modulith's
knoweb:
  role: ${KNOWEB_ROLE:web}  # web | worker | ingest
  business-timezone: Asia/Colombo
```

4.2 Module descriptor convention

Every module is a top-level package under `lk.coopfed.knoweb` with a `package-info.java` that names it and declares its allowed dependencies. Modulith derives the module graph from packages; the descriptor makes the intent explicit and reviewable.

```
// backend/app/src/main/java/lk/coopfed/knoweb/m1party/package-info.java
@org.springframework.modulith.ApplicationModule(
    displayName = "M1 Party, Tenancy & Security",
    allowedDependencies = { "kernel" } // layer 1 depends on layer 0 only
)
package lk.coopfed.knoweb.m1party;

// backend/app/src/main/java/lk/coopfed/knoweb/m4trading/package-info.java
@org.springframework.modulith.ApplicationModule(
    displayName = "M4 Trading Documents",
    allowedDependencies = { "kernel", "m1party::api", "m2catalogue::api", "m3pricing::api",
                          "m5inventory::query" } // layer 2 -> layer 1 api; sideways read-only
)
package lk.coopfed.knoweb.m4trading;
```

Sub-package	Visibility	Contains
.api	Published (named type api)	Command records, command handler interfaces, query interfaces, event records, value types. Nothing else.
.query	Published, read-only (named type query)	Query interfaces only; the sideways read-only surface of rule R2
.internal	Module-private	Entities, repositories, handlers, services, controllers, projections
.web	Module-private	REST controllers implementing the generated OpenAPI interfaces; web-facing DTO mapping

4.3 Kernel package as scaffolded by 17A

17A creates the kernel package with the interfaces every module codes against and **stub** implementations so the hello module runs; 19A replaces each stub with the real service. Modules never see the difference.

Interface (kernel.api)	Stub in 17A	Real in 19A
ScopeContext – current user, entity/location scopes, policy class	Fixed dev scope from a header	From the OIDC token; sets session variables
AuditFacade.record(...)	Inserts into kernel.audit_event without type validation	Catalogue-validated, correlation-aware
EventPublisher.publish(DomainEvent)	Inserts into kernel.event_outbox; no relay	Outbox + relay + broker + inbox
IdempotencyStore	In-memory map	kernel.idempotency_key table
ConfigRegistry.get(key, scope)	Reads seed defaults only	Layered resolution with cache
Messages.t(id, locale, args)	Loads i18n/*.json; no fallback marking	ICU MessageFormat; fallback marker
DocumentBase (entities for document, line, link, state history)	Entities and repository present; issuance protocol stubbed	Full issuance protocol and numbering
Clock / BusinessDate	System clock; business date = calendar date	Per-location day-close

4.4 Cross-cutting conventions

Aspect	Design
Commands and handlers	A command is an immutable record in api; its handler is a class in internal annotated @CommandHandler (kernel annotation) whose handle method runs in one transaction: guards → mutation → audit.record → events.publish. The kernel's interceptor supplies idempotency and permission checks from the annotation's permission attribute (<i>restated from doc 19 §3.4</i>).
Queries	Interfaces in query or api; implementations read through repositories; never return entities across the boundary, only records.
Events	Records implementing DomainEvent with a static TYPE constant in the dotted versioned form; payload fields are identifiers and changed facts only (<i>restated from doc 19 §6.1</i>).
Errors	RFC 9457 problem details; error codes are stable strings (relationship.overlap) and are message-catalogue ids so the web renders them in the user's language.
Logging	Structured JSON; every line carries correlation id, user, entity scope, module; no personal data or credential material in logs.
Time and money	Timestamps Instant UTC plus a LocalDateTime device reading where relevant; money BigDecimal scale 2; quantities scale 3 (<i>restated from doc 18 P8, P9</i>).
Identifiers	UUIDv7 via a kernel Ids.next(); natural keys are unique constraints, never primary keys.

Aspect	Design
Mapping	Entities stay inside the module; DTOs for the API are generated from the OpenAPI slice; a small mapper class per aggregate.

5. Boundary enforcement in the build

Rules R1–R7 of doc 17 §6.1 become tests that run on every commit. They are the only reason a modular monolith stays modular.

```
// backend/app/src/test/java/lk/coopfed/knoweb/ArchitectureTests.java
class ArchitectureTests {

    static final ApplicationModules modules = ApplicationModules.of(KnowebApplication.class);

    @Test void modulithStructureIsValid() {                // R1, R5: published packages, allowed dependencies
        modules.verify();
    }

    @Test void documentsTheModuleGraph() {                // keeps docs/modules/ current
        new Documenter(modules).writeDocumentation();
    }

    static final JavaClasses classes = new ClassFileImporter()
        .withImportOption(DO_NOT_INCLUDE_TESTS).importPackages("lk.coopfed.knoweb");

    @Test void layersAreRespected() {                    // R2, R3
        layeredArchitecture().consideringOnlyDependenciesInLayers()
            .layer("kernel").definedBy("..kernel..")
            .layer("master").definedBy("..m1party..", "..m2catalogue..", "..m3pricing..")
            .layer("transactions").definedBy("..m4trading..", "..m5inventory..", "..m6pos..",
                "..m7customers..", "..m10procurement..")
            .layer("read").definedBy("..m8reporting..", "..m9integration..")
            .whereLayer("master").mayOnlyAccessLayers("kernel")
            .whereLayer("transactions").mayOnlyAccessLayers("kernel", "master")
            .whereLayer("read").mayOnlyAccessLayers("kernel", "master", "transactions")
            .check(classes);
    }

    @Test void noCrossSchemaRepositories() {              // R4: a repository touches its own schema only
        classes().that().areAnnotatedWith(Repository.class)
            .should(new OwnSchemaOnly()).check(classes); // custom condition: @Table(schema=) matches module
    }

    @Test void commandsCarryPermissions() {              // R7: no handler without a permission
        classes().that().areAnnotatedWith(CommandHandler.class)
            .should(new HavePermissionAttribute()).check(classes);
    }

    @Test void kernelImportsNoModule() {                 // R5
        noClasses().that().resideInAPackage("..kernel..")
            .should().dependOnClassesThat().resideInAnyPackage("..m1party..", "..m2catalogue..", /* ... */
                "..hello..")
            .check(classes);
    }
}
```

Two further checks live in the pipeline rather than in Java: a schema-ownership script that parses each module's migrations and fails if a migration touches another module's schema, and an i18n check that fails the build when a message id lacks any of the three languages (*restated from doc 19 §5.2*).

6. Migration framework and database roles

6.1 Baseline migration (kernel location)

```
-- db/migration/kernel/V0001__baseline.sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ICU collations used by every *_si / *_ta / *_en column (doc 19 §5.4)
CREATE COLLATION IF NOT EXISTS si_icu (provider = icu, locale = 'si-LK');
CREATE COLLATION IF NOT EXISTS ta_icu (provider = icu, locale = 'ta-LK');
CREATE COLLATION IF NOT EXISTS en_icu (provider = icu, locale = 'en-LK');

-- one schema per module (doc 17 ADR-29)
CREATE SCHEMA IF NOT EXISTS kernel;      CREATE SCHEMA IF NOT EXISTS hello;
CREATE SCHEMA IF NOT EXISTS party;      CREATE SCHEMA IF NOT EXISTS security;
CREATE SCHEMA IF NOT EXISTS catalogue;  CREATE SCHEMA IF NOT EXISTS pricing;
CREATE SCHEMA IF NOT EXISTS trading;    CREATE SCHEMA IF NOT EXISTS inventory;
```

```

CREATE SCHEMA IF NOT EXISTS pos;          CREATE SCHEMA IF NOT EXISTS customers;
CREATE SCHEMA IF NOT EXISTS reporting;    CREATE SCHEMA IF NOT EXISTS integration;

-- roles (doc 18 P4, P10): the application never holds DDL, UPDATE or DELETE on ledgers
CREATE ROLE app_rw      NOLOGIN;  -- module code
CREATE ROLE app_relay NOLOGIN;    -- outbox relay: UPDATE published_at only
CREATE ROLE app_report NOLOGIN;   -- reporting projections and the analyst role: SELECT on replica
GRANT USAGE ON SCHEMA kernel, hello, party, security, catalogue, pricing, trading,
                    inventory, pos, customers, reporting, integration TO app_rw;

-- session scope variables read by RLS policies (doc 18 3.7); set by the kernel scope filter
-- app.scope_entity_id, app.scope_location_id, app.scope_class (custom GUCs, no DDL needed)

-- helper used by every RLS policy
CREATE OR REPLACE FUNCTION kernel.scope_entity() RETURNS uuid
    LANGUAGE sql STABLE AS $$ SELECT nullif(current_setting('app.scope_entity_id', true), '')::uuid $$;
CREATE OR REPLACE FUNCTION kernel.scope_location() RETURNS uuid
    LANGUAGE sql STABLE AS $$ SELECT nullif(current_setting('app.scope_location_id', true), '')::uuid $$;
CREATE OR REPLACE FUNCTION kernel.scope_class() RETURNS text
    LANGUAGE sql STABLE AS $$ SELECT coalesce(current_setting('app.scope_class', true), 'NONE') $$;

```

6.2 Conventions every module migration follows

Aspect	Design
Location and naming	db/migration//V_.sql; numbering is per module location (Flyway keeps one history table per schema when flyway.schemas is set per location group; 17A configures one Flyway instance per module location in code, see FlywayConfig.java).
Ownership	A migration touches only its module's schema; references to other schemas are by uuid columns without foreign keys (R4). The pipeline script enforces it.
Row-level security	Every operational table: ALTER TABLE ... ENABLE ROW LEVEL SECURITY; FORCE ROW LEVEL SECURITY; plus the policy template of Section 6.3. A table without a policy fails the schema test.
Grants	Ledgers and documents: INSERT only to app_rw (plus UPDATE on the columns the design allows, e.g. document.status). Reference tables: INSERT, UPDATE. No DELETE anywhere for app_rw.
Partitioning	Volume tables created partitioned by month on received_at with a kernel job creating partitions three months ahead (19A).
Expand then contract	A migration never drops or renames a column in use; add, backfill, switch code, drop in a later release (doc 14 §1.3 rolling deployment).

6.3 Row-level security policy template

```

-- applied to every operational table; classes from doc 18 3.7
ALTER TABLE party.location ENABLE ROW LEVEL SECURITY;
ALTER TABLE party.location FORCE ROW LEVEL SECURITY;

CREATE POLICY own_read  ON party.location FOR SELECT TO app_rw
    USING (owner_entity_id = kernel.scope_entity()
        AND (kernel.scope_location() IS NULL OR location_id = kernel.scope_location()));
CREATE POLICY own_write ON party.location FOR INSERT TO app_rw
    WITH CHECK (owner_entity_id = kernel.scope_entity());
CREATE POLICY fed_view  ON party.location FOR SELECT TO app_rw
    USING (kernel.scope_class() = 'FEDERATION_VIEW');
CREATE POLICY ext_view  ON party.location FOR SELECT TO app_rw
    USING (kernel.scope_class() = 'EXTERNAL_TIMEBOXED'
        AND owner_entity_id = ANY (kernel.granted_entities()));  -- function provided by 19A
-- PARTY policies exist only on document tables and use party columns (seller/buyer) - see 19A

```

7. Web shell scaffold

```
web/src/
main.tsx  App.tsx  router.tsx
shell/
  auth/      oidc.ts (authorization code + PKCE), session.ts, StepUpDialog.tsx
  scope/     ScopeContext.tsx, ScopeBanner.tsx, useScope.ts
  nav/       Navigation.tsx (route groups filtered by permissions)
  i18n/      messages.ts (ICU MessageFormat via FormatJS), useT.ts, LangFallbackTag.tsx
  api/       client.ts (fetch wrapper: bearer, idempotency key, problem-details, step-up retry)
  components/ DocumentHeader, DocumentLines, StateChip, Timeline, ApprovalBar, ReasonCapture,
              AttachmentCapture, ItemCard, MoneyDisplay, FreshnessStamp, ExceptionQueue, TrainingBadge
              tokens.css (type scale, 8px grid, semantic colours), fonts (Noto Sans, Sinhala, Tamil)
  design/
modules/
  hello/     routes.tsx, HelloPage.tsx, hello.messages.json
  m1party/ ... (added by each module)
generated/   hello.ts, m1party.ts ... (never edited; regenerated by tools/gen-clients.sh)
```

Aspect	Design
Module registration	Each module exports a <code>ModuleDefinition { id, routes, navItems, requiredPermissions }</code> ; <code>router.tsx</code> assembles them. A module developer adds a folder and a line, nothing in the shell.
Generated clients	<code>tools/gen-clients.sh</code> runs <code>openapi-typescript</code> over every slice in <code>backend/app/src/main/resources/openapi/</code> into <code>web/src/generated/</code> ; CI fails if generated output is stale.
Data fetching	TanStack Query with stale-while-revalidate; every list server-paged; mutations send an idempotency key and handle the step-up challenge (HTTP 401 with <code>step_up_required</code>) by opening <code>StepUpDialog</code> and retrying.
i18n	Message ids, three JSON catalogues, ICU plurals; <code>useT</code> returns the string and a fallback flag so components can render the EN tag (<i>restated from doc 30 §2.1</i>).
Testing	Vitest for components and hooks; Playwright smoke test that logs in, sees the scope banner, opens the hello screen in each language.
Build	Vite; content-hashed assets; <code>VITE_API_BASE</code> ; the shell reads the API's advertised version and prompts reload on mismatch (<i>doc 31 §7</i>).

8. Till shell scaffold

```
till/
app/
  src/main/java/lk/coopfed/knoweb/till : Application, MainActivity (lock task), navigation, DI
  ui/
    data/
      # Compose: SalesScreen skeleton (zones of doc 30 4.3), PinScreen, HelloScreen
      # Room database: kernel tables mirror (document, line, state_history, movement,
      #   audit_event, counter, outbox) + cache tables (snapshot_*)
    engine/
      # adapter to shared-engine
  core/
    peripherals/
      # ids (UUIDv7), clock/offset, business date, money, i18n loader, fonts
      # interfaces per device class + Simulator implementations (doc 30 4.2)
      # outbox writer, upload worker (WorkManager), snapshot applier - stubs in 17A, 26A
  sync/
    fills
  agent-test/
    # tiny device-owner app used to validate provisioning and silent install (doc 31)
```

Aspect	Design
Stack	Kotlin 2, Jetpack Compose, Room with SQLCipher, WorkManager, Hilt; <code>minSdk 30</code> ; the shared engine as a Gradle dependency on <code>:shared-engine</code> (<i>restated from doc 30 §4.1, doc 14 §2</i>).
Kiosk	<code>MainActivity</code> requests lock-task mode; a debug flag disables it on developer devices; the device-owner configuration itself is the management agent's job (<i>doc 31</i>).
Database	Room entities generated from the same logical model as <i>doc 18</i> parts C–E; <code>PRAGMA journal_mode=WAL</code> ; <code>synchronous=FULL</code> on issuance transactions; encryption key in the Android keystore.
Fonts	Noto Sans, Noto Sans Sinhala, Noto Sans Tamil bundled; a <code>ReceiptRenderer</code> stub that draws a receipt to a bitmap and saves it (the printer adapter arrives with 26A).
Hello screen	Reads a hello row from the local cache, calls the shared engine's stub resolver, shows the result in the shop language.
Testing	Unit tests on JVM; instrumentation smoke test on an emulator in CI; the power-loss harness for spike 1 lives under <code>till/app/src/androidTest/</code>

9. Shared engine library

ADR-31: one library, no server dependencies, used by the backend and the till so a bill computes identically offline and online (*restated from doc 17 §3 P7*). 17A creates the module with its public interfaces and stub implementations; M3 (23/23A) supplies the real price resolution and rule evaluation, the kernel (19A) the envelope and counter.

```
backend/shared-engine/src/main/kotlin/lk/coopfed/knoweb/engine/  
Money.kt           // BigDecimal scale 2, rounding rules (cash to rupee, exact otherwise)  
Quantity.kt        // scale 3, unit conversion by factor  
Tax.kt             // inclusive-price tax decomposition from a rate  
PriceResolver.kt   // interface: resolve(listPrice, batchMrps, controlPrice, policy) -> Resolution  
RuleEvaluator.kt   // interface: evaluate(basket, rules) -> Benefits (closed vocabulary, doc 23)  
Envelope.kt        // event envelope data class + canonical serialisation + hash  
Counter.kt         // gapless counter contract: next(series) with persistence callback  
BusinessDate.kt    // day-close semantics
```

Aspect	Design
Purity	No Spring, no Android, no I/O; persistence is injected as callbacks. A unit test suite in the library is the single source of truth for numeric behaviour.
Versioning	The library version is stamped into every snapshot and every upload (engine_version in the envelope) so central can re-validate with the same version the till used (<i>restated from doc 32 §5.2</i>).
Consumption	The backend depends on it as a project module; the till as a Maven artefact published by CI (same coordinates, same version).

10. Local environment

```
# infra/compose/docker-compose.yml (excerpt)
services:
  postgres:
    image: postgres:16
    # built with ICU; POSTGRES_INITDB_ARGS="--locale-provider=icu --icu-locale=en
  pgbouncer:
    image: edoburu/pgbouncer
  rabbitmq:
    image: rabbitmq:3.13-management
  minio:
    image: minio/minio
    # buckets: attachments, images, packages, audit-anchors (object lock)
  keycloak:
    image: quay.io/keycloak/keycloak:25
    # imports infra/compose/realms-dev.json (users: fed-admin, mpcs-admin, cashier)
  mailpit:
    image: axllent/mailpit # catches notification e-mails
  backend:
    image: built by jib; KNOWEB_ROLE=web,worker (both profiles in one dev container)
  web:
    vite dev server
```

Aspect	Design
make up	Starts everything, waits for health, runs migrations and seeds, prints the URLs and the three dev logins.
Seeds	Dev realm with three users in the three languages; the hello module's sample rows; permission catalogue and templates arrive with 19A/21A seeds.
Two instances	make up-2 starts two backend containers behind a tiny nginx to catch instance-affinity bugs from day one (<i>restated from doc 14 §1.3</i>).
Reset	make reset drops volumes; the fifteen-minute test of the cover is run against a fresh clone in CI nightly.

11. Continuous integration

Stage	Runs	Fails on
build	Gradle build (backend, shared engine), pnpm build (web), Gradle assemble (till)	Compile errors; stale generated clients; stale Modulith documentation
verify	ArchitectureTests; schema-ownership script; i18n completeness; lint and format	Any boundary or ownership violation; missing translation id
test	Unit tests; integration tests with Testcontainers (PostgreSQL with RLS, RabbitMQ, MinIO, Keycloak); web Vitest; till JVM tests	Any failure; coverage below the module's floor (set per module in nA)
e2e	Playwright smoke against the composed stack with two backend instances; Android emulator smoke	Login, scope, hello flow failures
package	Jib image (backend), static bundle (web), APK signed with the dev key, shared-engine artefact, SBOM (CycloneDX)	–
publish (main only)	Push images and artefacts to the registry; tag with semantic version + build number	–

Release signing with the Federation key, ring deployment and the update service belong to doc 31 and doc 35; 17A stops at a signed dev build and published artefacts.

12. The hello module: the template every module copies

A deliberately trivial module that exercises every kernel interface and every convention, so that copying it gives a module developer a working skeleton and the reviewers a reference for what “done like the others” means.

```
lk.coopfed.knoweb.hello/
package-info.java           @ApplicationModule(allowedDependencies = {"kernel"})
api/ RegisterGreeting.java   record(entityId, textEn, textSi, textTa) -- the command
    GreetingRegistered.java  record implements DomainEvent TYPE = "hello.greeting.registered.v1"
    GreetingQueries.java     interface: find(id), list(scope)
internal/ Greeting.java      @Entity @Table(schema="hello") owner_entity_id, three names, status
    GreetingRepository.java
    RegisterGreetingHandler.java @CommandHandler(permission = "hello.greeting.register")
    GreetingQueriesImpl.java
web/ HelloController.java    implements generated HelloApi (openapi/hello.yaml)
db/migration/hello/V0001__greeting.sql table + RLS policies from 6.3 + INSERT-only grant
openapi/hello.yaml          POST /v1/hello/greetings, GET /v1/hello/greetings/{id}
i18n/*.json                 hello.title, hello.register, hello.registered
web/src/modules/hello/      route, HelloPage using DocumentHeader-style card + ScopeBanner
till/app/.../ui/HelloScreen.kt reads cache, shows in shop language

// RegisterGreetingHandler.java - the shape of every handler in the system
@CommandHandler(permission = "hello.greeting.register")
```

```

class RegisterGreetingHandler implements Handles<RegisterGreeting, GreetingId> {

    @Transactional
    public GreetingId handle(RegisterGreeting cmd, ScopeContext scope) {
        // 1. guards (business rules) - throw ProblemException("hello.greeting.duplicate") etc.
        // 2. mutation
        Greeting g = Greeting.create(Ids.next(), scope.entityId(), cmd.textEn(), cmd.textSi(), cmd.textTa());
        repo.save(g);
        // 3. audit in the same transaction
        audit.record(AuditType.HELLO_GREETING_REGISTERED, Subject.of(g), null, g.snapshot(), scope);
        // 4. event in the same transaction (outbox)
        events.publish(new GreetingRegistered(g.id(), scope.entityId()));
        return g.id();
    }
}

```

What the hello module proves	How
Boundary tests pass with a real module present	ArchitectureTests
Migration, schema, RLS and grants work	Integration test: a user scoped to entity A cannot read entity B's greeting; INSERT allowed, UPDATE denied
Command pipeline: permission, idempotency, audit, outbox in one transaction	Integration test: resend with the same idempotency key returns the same id; one audit row; one outbox row
OpenAPI-first controllers and generated web client	The web hello page posts a greeting and lists it
i18n end to end	The page and the till screen render in the three languages with the EN fallback tag when a translation is missing
Shared engine reachable from both sides	Backend and till call Money.roundCash and get the same result in tests
Two instances	Playwright run alternates instances; nothing breaks

13. The fifteen-minute start (README contents)

```

git clone <repo> && cd knoweb
make up          # ~10 min first time: pulls images, migrates, seeds; prints http://localhost:8080 and :5173
open http://localhost:5173 -> login fed-admin / dev -> Hello -> register a greeting
make test        # unit + architecture tests (~2 min)
make test-int    # integration with Testcontainers (~6 min)
make gen-clients # after changing an OpenAPI slice
make new-module NAME=m2catalogue SCHEMA=catalogue # copies hello with renames (Section 12)

```

14. Sprint 0 tickets (17A part)

#	Ticket	Est. (days)	Done when
S0-01	Repository, Makefile, version catalogue, README skeleton	0.5	make up prints URLs on an empty stack
S0-02	Backend Gradle project, Spring Boot app, three profiles, health and metrics	1	App starts under each profile
S0-03	Kernel package with api interfaces and stubs (Section 4.3)	1.5	Hello handler compiles against them
S0-04	Flyway per-module configuration, baseline migration, roles, collations, RLS helpers	1	Fresh DB migrates to baseline; schema list matches
S0-05	ArchitectureTests and pipeline scripts (schema ownership, i18n completeness)	1	A deliberate violation fails the build
S0-06	OpenAPI-first toolchain: generator config, hello.yaml, controller stub, gen-clients script	1	Client generated; stale check works
S0-07	Web shell: auth (PKCE against dev realm), scope context and banner, module registration, i18n loader, design tokens, component seeds	3	Login → banner → hello page in three languages
S0-08	Till shell: project, Compose skeleton, Room mirror tables, fonts, hello screen, simulator peripherals	3	APK builds; hello screen on emulator; JVM tests
S0-09	Shared engine module with interfaces, stubs and tests; published artefact	1	Consumed by backend and till
S0-10	Local environment compose with all services, dev realm, seeds, two-instance variant	1.5	make up and make up-2 pass
S0-11	CI pipeline: build, verify, test, e2e, package, publish	2	Green on main; nightly fresh-clone test
S0-12	Hello module complete with integration tests (Section 12)	1.5	All rows of the proof table pass
S0-13	make new-module scaffolder	0.5	Copies hello with renames; boundary tests still pass

Total about 18.5 developer-days: nine working days for the platform pair, matching Sprint 0 weeks 1–2. 19A's tickets run in the same sprint from day 4, replacing stubs as they land.

15. Open points taken as assumptions

Point	Assumed in 17A	Source
Broker product	RabbitMQ in compose and CI; the kernel publisher is behind an interface so NATS is a swap	Doc 19 DR-2, doc 14 DR-2
Identity provider	Keycloak in compose; the wrapper interface in kernel.api hides it	Doc 19 DR-1
CI host	GitHub Actions syntax shown; GitLab CI equivalent is mechanical	DevOps
Single Gradle module for the backend vs one per module	Single module with Modulith packages: simpler build, same enforcement; extraction later is a package move	Platform pair
Web component base	Headless primitives (Radix-class) styled by tokens	Doc 30 DR-2
Terminal model for the till emulator profile	Generic 8-inch 1280×800 profile until doc 14 DR-3 closes	Hardware procurement

End of document. 17A v0.1; superseded in detail by the repository itself once Sprint 0 completes – the README and the hello module are then the living version of this guide.